



University
of Basel

Operating Systems

Spring Semester 2026

Project Ideas

Prof. Dr. F. Ciorba

Dr. O. Simsek

T. Jakobsche



hpc.dmi.unibas.ch



Overview

1	Supervisors
2	Timeline
3	Requirements
4	Templates and Guidelines
5	Project Ideas
6	Next Steps

Project Supervisors

Always send your emails to both.

Dr. Osman Simsek

osman.simsek@unibas.ch



Dr. Thomas Jakobsche

thomas.jakobsche@unibas.ch



Overview

1 Supervisors

2 **Timeline**

3 Requirements

4 Templates and Guidelines

5 Project Ideas

6 Next Steps

Project Timeline Spring 2026

March 6, 13:15 Introduction to project ideas

- Form teams of **four** students and choose a topic.
- Write a short project proposal (description and requirements).
- Hand-in your project proposal as PDF on ADAM.

March 13, 16:00 Submit project proposals on ADAM **[deadline]**

- Receive feedback and approval from project supervisors until **March 18**.
- Create your project idea presentation and upload it as PDF with the correct name on ADAM.

March 26, 10:00 Submit idea presentations on ADAM **[deadline]**

March 26-27 Project idea presentations (7 mins + 3 mins Q&A)

- Work on your projects. If necessary, there will be feedback meetings with supervisors.
- Prepare the project deliverables: final presentation, 3-5 pages report, artifacts (e.g., code + readme)
- Upload a zip file with the correct name on ADAM.

June 10, 16:00 Submit final deliverables on ADAM **[deadline]**

June 11-12 Present your project (12 mins + 3 min Q&A)

Overview

1	Supervisors
2	Timeline
3	Requirements
4	Templates and Guidelines
5	Project Ideas
6	Next Steps

Project Requirements

Mandatory requirements for all team members of project groups

- Give and attend all **initial presentations**
- Give and attend all **final presentations**
- Hand in the **project deliverables** (presentation, report, artifacts)
- Every team member must **contribute fairly** to the project
- Follow DMI's "**AI in Teaching**" section, with "**Guidelines/Leitfaden**" and the "**Guidance/Wegleitung**":
<https://dmi.unibas.ch/de/studium/computer-science-informatik/dokumente/>
with the "**Wegleitung**" in English
https://dmi.unibas.ch/fileadmin/user_upload/dmi/Studium/Computer_Science/Diverses/Wegleitungen/Dokumente_2023/Guidance_on_AI_in_teaching_in_Computer_Science.pdf

Not fulfilling **all** these requirements leads to failing the project (40% of the overall OS course grade) **and** the course.

Hand-in through ADAM > Group Projects

- > **Initial Presentations** as PDF
- > **Final Presentations + Reports + Code + Artifacts** as zip

Project Grading

Overall **OS course grade** is the weighted average of the two mandatory parts of the course:

Written exam grade: **Weight 60%**

Project grade: **Weight 40%**

OS projects are **graded** according to 3 criteria:

Execution: Degree of **complexity** of idea and its implementation

Report: **Description** of project execution

Presentation: Final **presentation & demo** (initial presentation not graded)

Team project work, but **individual grading for each student**.

→ Document and clarify during final presentation and report who worked on which parts, with a fair distribution of the work among the project members.

Overview

1	Supervisors
2	Timeline
3	Requirements
4	Templates and Guidelines
5	Project Ideas
6	Next Steps

Heilmeier Catechism of George H. Heilmeier

Former DARPA (Defense Advanced Research Projects Agency) Director

Created by former DARPA director George H. Heilmeier, the catechism is a set of eight questions that researchers and engineers answer to fully understand the context, costs, and effort required to execute a given project.

<https://www.darpa.mil/about/heilmeier-catechism>

1. What are you trying to do? Articulate your objectives using absolutely no jargon.
2. How is it done today, and what are the limits of current practice?
3. What is new in your approach and why do you think it will be successful?
4. Who cares? If you are successful, what difference will it make?
5. What are the risks?
6. How much will it cost?
7. How long will it take?
8. What are the mid-term and final “exams” to check for success?

Presentation Guidelines of the AD Group

Algorithms and Data Structures at University of Freiburg

Created by the AD Teaching Group at the University of Freiburg, these guidelines provide a structured framework and practical advice for delivering clear, engaging, and visually accessible academic presentations.

<https://ad-wiki.informatik.uni-freiburg.de/teaching/PresentationGuidelines>

- **Structure chronologically:** Problem, Solution, Evaluation.
- **Teach by example:** Show examples before formal definitions.
- **Visualize, don't list:** Use visuals and animations instead of dense text.
- **Prepare the environment:** Arrive early to test equipment and demos.
- **Rehearse strategically:** Memorize your opening and practice tricky explanations.
- **Walk through everything:** Explain every visual completely or omit it.
- **Keep it readable:** Minimize text, use large fonts (18pt+), and ensure high contrast.
- **Less is more:** Don't rush; move extra details to backup slides.

Writing Guidelines of the AD Group

Algorithms and Data Structures at University of Freiburg

Created by the AD Teaching Group at the University of Freiburg, these guidelines outline five "Golden Rules" to help students and researchers produce clear, unambiguous, and highly readable scientific texts.

<https://ad-wiki.informatik.uni-freiburg.de/teaching/WritingGuidelines>

- **Simple sentences:** Use a basic subject-predicate-object structure; split complex sentences with multiple verbs.
- **Well-defined terms:** Always define unfamiliar concepts or specific variants before using them.
- **Consistency:** Stick to the exact same terminology and visual formatting throughout the text.
- **Be concrete:** Avoid vague language; provide precise, specific details to demonstrate understanding.
- **Clear context:** Ensure every sentence has a clear logical connection to the surrounding text.

Overleaf Templates

Use the Overleaf templates for presentations and report provided on

ADAM > Projects >

- OS-project-presentation-template
- OS-project-report-template

See next slides

- Initial presentation content
- Final presentation content
- Report content

Initial Presentation

Mandatory but not graded

- **Title Slide:** Project title, group members, course name, date, etc.
- **Problem Statement:** Introduce the problem your project is trying to solve, motivate why solving this problem is important, highlight the significance of solving this problem. (What are you trying to do and why?)
- **Proposed Solution:** How do you plan to solve this problem? Will you require any hardware to do so (e.g., Raspberry Pi's)? Will you require any third-party software or libraries?
- **Preliminary Time and Work Plan:** Provide a preliminary time plan (e.g., Gantt chart in a readable form) and division of work among group members.

Final Presentation

Mandatory and graded

- **Title Slide:** Project title, group members, course name, date, etc.
- **Problem Statement:** Introduce the problem you are trying to solve, motivate why solving this problem is important, highlight the significance of solving this problem. (What are you trying to do and why?)
- **Implemented Solution:** How did you solve the problem? Prioritize a conceptual explanation of implementation, configuration, and setup. (May use code snippets to explain concepts but not full code listings.) Did you use any hardware (e.g., Raspberry Pies)? Did you require any third-party software or libraries? How did you divide the work among group members?
- **Live Demonstration:** Show the main functionality of your problem and solution in a live demonstration. *(Hint: Prepare a recording just in case.)*
- **Discussion and Conclusion:** How do you evaluate the achieved results? Did you meet your initial time plan? Are there any open TODOs?
- **Lessons Learned:** Information about what you learned, regarding software development, acquired skills, time management, working in a team, etc.

Project Report

Mandatory and graded

Title Page: Project title, group members, course name, date, etc.

1. **Introduction:** Introduce the problem you are trying to solve and the necessary context, motivate why solving this problem is important, highlight the significance of solving this problem. (What are you trying to do and why?)
2. **Background:** Introduces relevant background knowledge to understand your work (e.g., existing algorithms, protocols, software, and technologies).
3. **Methodology:** Conceptual explanation of implementation, configuration, and setup. (May use code snippets and screenshots to explain concepts, but not full code listings.)
4. **Results:** Show the result/capabilities of your solution through plots, tables, screenshots, videos, etc.
5. **Discussion:** Discuss the achieved results (good and bad), their limitation(s) and significance for solving the problem.
6. **Conclusion:** Summarize what has been achieved, what is left open from your initial plan, and how your solution could be further developed/improved through future work.

Lessons Learned: What did you learn during the project? Which skills did you acquire? What did you learn about yourself and your role in a software development team?

References: List all references using proper citations, including third-party software and libraries (check software release licenses about usage permissions).

Appendix: *Declaration of Independent Authorship* (see next slides), declaration of usage of AI, and any additional information (e.g., plots, tables, screenshots) that did not make it into the report (or results section) but is still relevant.

Declaration of Independent Authorship

Per project team, include a “[Declaration of Independent Authorship/Eigenständigkeitserklärung](#)” in your report, **signed** by each member of the project team

<https://dmi.unibas.ch/de/studium/computer-science-informatik/dokumente/>

[https://dmi.unibas.ch/fileadmin/user_upload/dmi/Studium/Computer Science/Diverses/Verwendung_AI/2025-02-17 Eigenstaendigkeitserklaerung Declaration-of-Independent-Authorship DE EN neu.pdf](https://dmi.unibas.ch/fileadmin/user_upload/dmi/Studium/Computer_Science/Diverses/Verwendung_AI/2025-02-17_Eigenstaendigkeitserklaerung_Declaration-of-Independent-Authorship_DE_EN_neu.pdf)



Eigenständigkeitserklärung (Februar 2025)

Im Zusammenhang mit KI / ChatGPT in der Lehre wurde vom Rechtsdienst der Universität Basel eine gesamtuniversitäre Formulierung erarbeitet. Sie kann für schriftliche Arbeiten auf allen Stufen des Studiums verwendet werden.

«Ich bezeuge mit meiner Unterschrift, dass ich meine Arbeit selbständig ohne fremde Hilfe verfasst habe und meine Angaben über die bei der Abfassung meiner Arbeit benutzten Quellen in jeder Hinsicht der Wahrheit entsprechen und vollständig sind. Alle Quellen, die wörtlich oder sinngemäss übernommen wurden, habe ich als solche gekennzeichnet.

Des Weiteren versichere ich, sämtliche Textpassagen, die unter Zuhilfenahme KI-gestützter Programme verfasst wurden, entsprechend gekennzeichnet sowie mit einem Hinweis auf das verwendete KI-gestützte Programm versehen zu haben.

Eine Überprüfung der Arbeit auf Plagiate und KI-gestützte Programme – unter Einsatz entsprechender Software – darf vorgenommen werden. Ich habe zur Kenntnis genommen, dass unläuteres Verhalten zu einer Bewertung der betroffenen Arbeit mit einer Note 1 oder mit «nicht bestanden» bzw. «fail» oder zum Ausschluss vom Studium führen kann.

Ich habe zur Kenntnis genommen, dass ich bei begründetem Verdacht auf eine unerlaubte oder nicht gekennzeichnete Anwendung von KI bei schriftlichen Leistungsüberprüfungen auf Einladung hin verpflichtet bin, an der Klärung des Verdachts mitzuwirken, z.B. durch Teilnahme an einem Gespräch.»

Diese Formulierung wird, wie weitere wichtige Entwicklungen mit gesamtuniversitärer Relevanz, über die laufend aktualisierte Intranet-Seite bereitgestellt: <https://intranet.unibas.ch/x/wo65CQ>

Declaration of Independent Authorship (February 2025)

In the context of AI / ChatGPT in teaching, the Legal Services Team of the University of Basel has prepared an adapted version of the Declaration of Independent Authorship for student essays and theses that can be used at all levels of study.

"I attest with my signature that I have completed this paper independently and without any assistance from third parties and that the information concerning the sources used in this paper is true and complete in every respect. All sources that have been quoted or paraphrased have been referenced accordingly.

Additionally, I affirm that any text passages written with the help of AI-supported technology are marked as such, including a reference to the AI-supported program used.

This paper may be checked for plagiarism and use of AI-supported technology using appropriate software. I understand that unethical conduct may lead to a grade of 1 or "fail" or to expulsion from the course of studies.

I have taken note of the fact that in the event of a justified suspicion of the unauthorized or undisclosed use of AI in written performance assessments, I am upon request obligated to cooperate in confirming or ruling out the suspicion, for example by attending an interview".

This formulation, as with other important developments of relevance to the university as a whole, is provided on the continuously updated intranet page: <https://intranet.unibas.ch/x/wo65CQ>

Overview

1	Supervisors
2	Timeline
3	Requirements
4	Templates and Guidelines
5	Project Ideas
6	Next Steps



University
of Basel

OS FS26 Project Ideas

1. Raspberry Pi OS Development

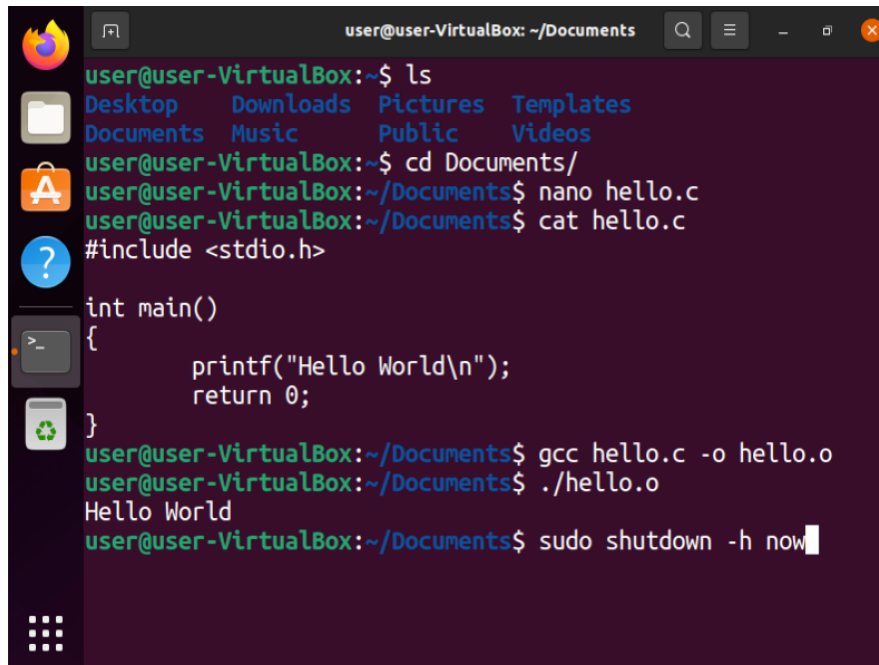
Design and implement a simple operating system for the Raspberry Pi. The project focuses on understanding the boot process, memory management, and device drivers. This hands-on project will help you understand low-level OS functionalities and hardware interactions.



Lessons Learned:
Start with a minimal bootloader
and incrementally add features
instead of attempting a full OS.

2. Custom Linux Shell

This project involves creating a custom Linux shell using C which also recognizes your mouse and/or touchscreen. Implement features such as command execution, signal handling, and piping. The project emphasizes understanding the Linux process model and system calls, enhancing your understanding of OS concepts.



```
user@user-VirtualBox: ~/Documents
user@user-VirtualBox:~$ ls
Desktop  Downloads  Pictures  Templates
Documents Music    Public   Videos
user@user-VirtualBox:~$ cd Documents/
user@user-VirtualBox:~/Documents$ nano hello.c
user@user-VirtualBox:~/Documents$ cat hello.c
#include <stdio.h>

int main()
{
    printf("Hello World\n");
    return 0;
}
user@user-VirtualBox:~/Documents$ gcc hello.c -o hello.o
user@user-VirtualBox:~/Documents$ ./hello.o
Hello World
user@user-VirtualBox:~/Documents$ sudo shutdown -h now
```

Lessons Learned:
Start with command execution,
piping, signal handling, and
command flag behavior. Leave
mouse/touch for last.

3. System Call Tracer and Profiler Tool

Develop a tool to trace and profile the usage of system calls in Linux using C.

You will intercept and log system calls made by programs and collect the frequency of system calls instances, understanding the role and mechanisms of system calls in OS operations.

Example trace for “ls”

```
execve("/usr/bin/ls", ["ls"], 0x7ffe08c8d100 /* 65 vars */) = 0
brk(NULL)                               = 0x17c3000
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x2b7a3
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
open("/opt/remi/php74/root/usr/lib64/tls/x86_64/libselinux.so.1", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
stat("/opt/remi/php74/root/usr/lib64/tls/x86_64", 0x7ffd0921de20) = -1 ENOENT (No such file or directory)
open("/opt/remi/php74/root/usr/lib64/tls/libselinux.so.1", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
stat("/opt/remi/php74/root/usr/lib64/tls", {st_mode=S_IFDIR|0555, st_size=6, ...}) = 0
open("/opt/remi/php74/root/usr/lib64/x86_64/libselinux.so.1", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
stat("/opt/remi/php74/root/usr/lib64/x86_64", 0x7ffd0921de20) = -1 ENOENT (No such file or directory)
open("/opt/remi/php74/root/usr/lib64/libselinux.so.1", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
```

Lessons Learned:
Start with a CLI tool that logs and summarizes syscall usage (ptrace or kernel module). Leave GUI for last.

4. Modify the OS Kernel

Make changes to the Linux kernel to add new system calls and functionality. The new system calls could perform a unique task, such as a specialized file operations or encryption/decryption utilities directly accessible from the kernel. You will gain hands-on experience with kernel development, compiling, and testing.

- > `whoami`: Display current user.
- > `ls`: List files and directories.
- > `cd`: Change directory.
- > `pwd`: Print the current working directory.
- > `cp`: Copy files or directories.
- > `mv`: Move or rename files/directories.
- > `rm`: Remove/delete files or directories.
- > `mkdir`: Create a new directory.
- > `cat`: Display the contents of a file.
- > `nano/vi`: Text editors for creating/editing files.
- > `chmod/chown`: Change file permissions/owner.
- > `ps/kill`: Display/terminate active processes.
- > `ssh`: Connect to a remote server.

Lessons Learned:
Recompiling the full kernel takes hours,
consider loadable kernel modules.

```
root@dmidec24:~# Thu Feb 27 13:34:47 on console  
root@dmidec24:~# mv *.ppt *.pptx
```

5. Process Monitoring and Management

Develop a tool to monitor and manage system processes. Using C and system calls, the project covers process creation, termination, resource allocation, and control resource usage limits. It aims to provide practical insights into process management and activity monitoring in operating systems.

```
Processes: 566 total, 4 running, 562 sleeping, 3192 threads
Load Avg: 3.60, 2.77, 2.77 CPU usage: 4.12% user, 12.98% sys, 82.88% idle SharedLibs: 208M resident
MemRegions: 227512 total, 4525M resident, 152M private, 1624M shared. PhysMem: 16G used (30% free)
VM: 5023G vsize, 1995M framework vsize, 2630243(0) swapins, 2945865(0) swapouts. No swap space used.
Disks: 18750548/148G read, 2352496/48G written.
```

PID	COMMAND	%CPU	TIME	#TH	#WQ	#PORT	MEM	PURG	CMPRS	PGRP		PPID	PPARG	UID
0	kernel_task	17.9	80:50.30	260/4	0	0	326M	0B	0B	0		0	0	0
43100	top	16.8	00:01.55	1/1	0	27	7640K+	0B	0B	43100		0	0	0
43885	texstudio	10.5	38:06.72	16/1	3	289+	152M	11M	119M	43885		0.74233	0.00000	67008
294	WindowServer	5.5	97:35.11	11	5	5163	966M	9600K	204M	294		0.00000	0.00070	88
35254	zoom.us	3.0	01:59:51	33	3	1566	306M	8192B	191M	35254	1	spring *0[1308]	0.00000	67008
1080	Activity Mon	2.3	22:21.68	6/2	4/1	1148	85M+	0B	51M	1080	1	running 0[1168]	0.00000	67008
259	airportd	0.8	03:57.96	11	9	204	13M+	0B	3316K	259	1	sleeping *5760[1]	0.00000	0
1823	Google Chrom	0.7	30:08.53	41	2	1324	439M	32K	110M	1823	1	sleeping *0[26758]	0.00000	67008
1236	Microsoft Ou	0.7	08:01.95	17	6	1670-	215M-	48M	90M	1236	1	sleeping 0[2906]	0.00000	67008
156	JamfDaemon	0.7	08:30.73	6	2	84	56M	0B	40M	156	1	sleeping *0[1]	0.00000	0
6712	Terminal	0.5	00:43.38	14	7	418	65M+	14M	30M	6712	1	sleeping *0[1155]	0.00070	67008

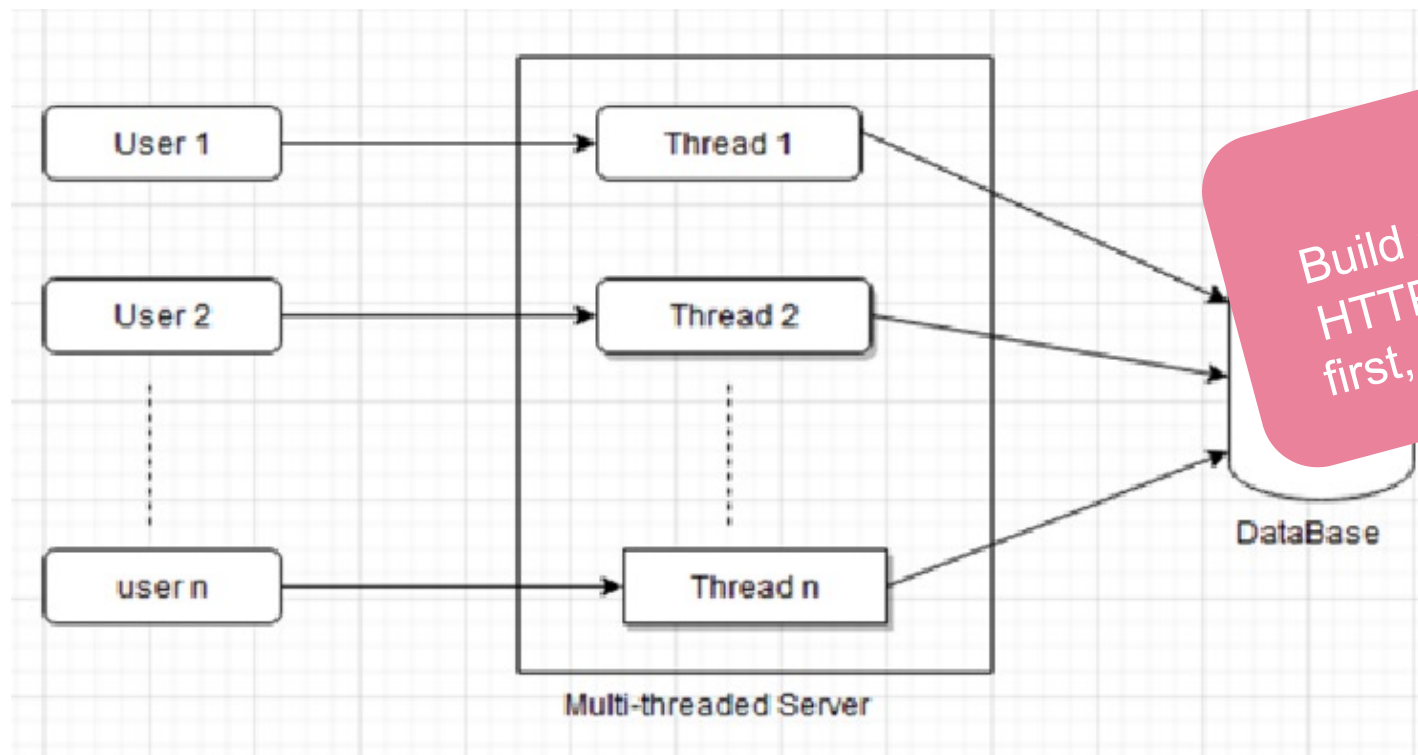
sharedLibs: 208M resident
16G used (300M free)
s. Net

Lessons Learned:
The /proc filesystem is your main data source. Start by parsing /proc/[pid]/stat and /proc/[pid]/status.

			PPID	PID	PPID	PID	UID
0.000000	0.000000	0					
0.000000	0.000000	0					
0.74233	0.00000	67008					
0.00000	0.00000	0					

6. Chat Room: Multi-Client to a Multi-threaded Server

Develop a multi-threaded web server in C, to which multiple clients can connect and chat with each other. The project covers thread creation, synchronization, and socket programming, emphasizing the concurrency model and network communication in operating systems.



Lessons Learned:
Build a multi-threaded TCP server (not HTTP). Correct concurrent messaging first, features like separate rooms later.

7. Modify the OS Scheduler

Modify an existing OS scheduler to implement priority-based and user-favoritism scheduling. The project focuses on understanding scheduling algorithms, process priorities, and their impact on system performance. Use C and modify the OS kernel code.

Process	Arrival Time	Burst Time	Priority	User Category
P1	0	11	2	Regular User
P2	5	28	0	High Priority
P3	12	2	3	Regular User
P4	2	10	1	Regular User
P5	9	16	4	Regular User

Lessons Learned:
Use Buildroot to compile a minimal Linux kernel and test with QEMU. Add a new scheduling policy to the CFS and compare process execution times.

8. Memory Allocation Problem Detector

In this project you will develop your own memory allocation detector in C. Detect memory allocation problems and trigger warning alerts when those become critical. In this project you will learn about memory management and common pitfalls of memory allocation.



Memory

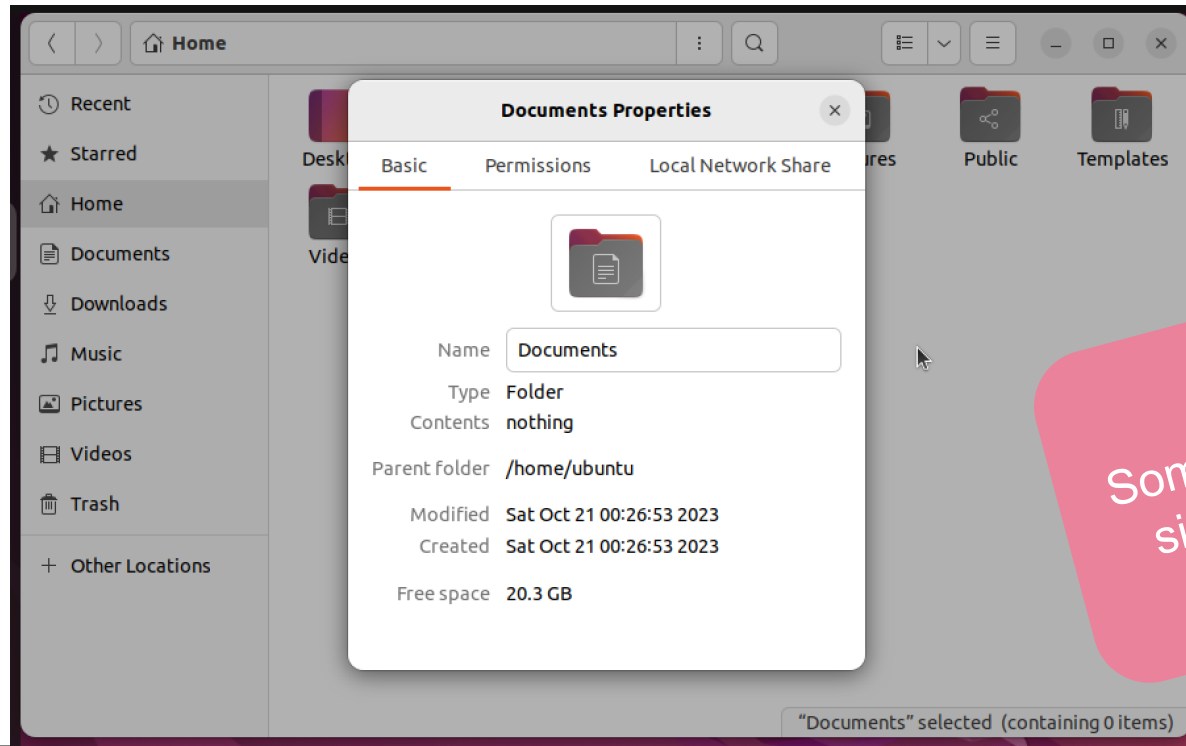
Lessons Learned:
You can use LD_PRELOAD to intercept
malloc/free/realloc without modifying the
target program's source code
Start with detecting leaks, then double-free,
then buffer overflow, etc.

[CodeReliant.io](https://www.codereliant.io)



9. File System Property Extension

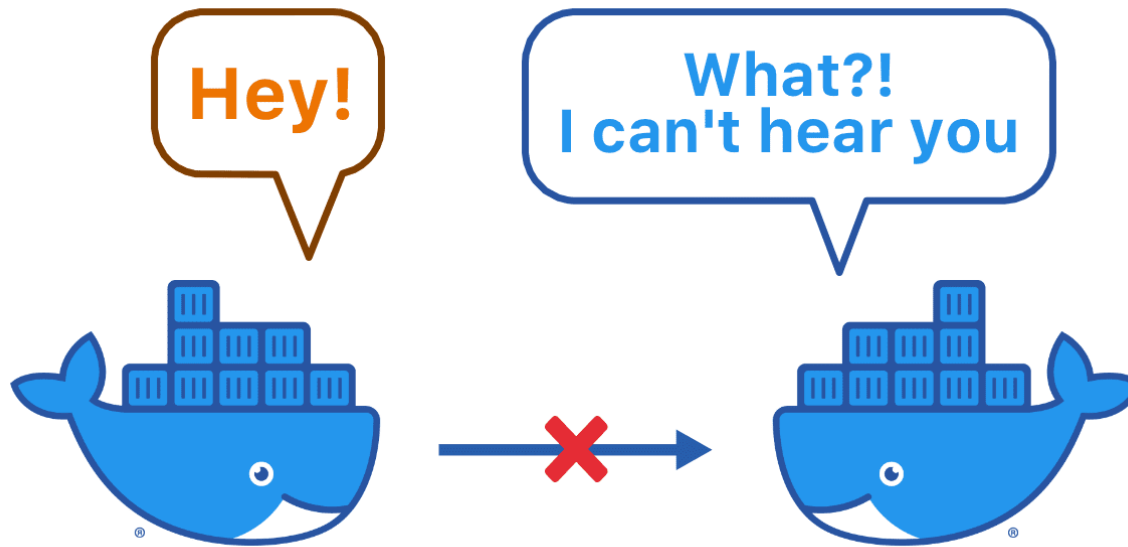
Extend a file system to include a new persistent properties for files or directories, such as tagging and provenance (e.g., downloaded or received by mail). The project covers file system architecture, metadata management, and system calls related to file operations.



Lessons Learned:
Some options: Linux extended attributes (xattrs), a
sidecar JSON database, or FUSE-based virtual
filesystem. Start with CLI, GUI later.

10. Container Communication via Ports

Set up a Docker container environment and enable communication through ports. You will learn about containerization, port mapping, inter-container communication, and understand the basics of networked application communication.



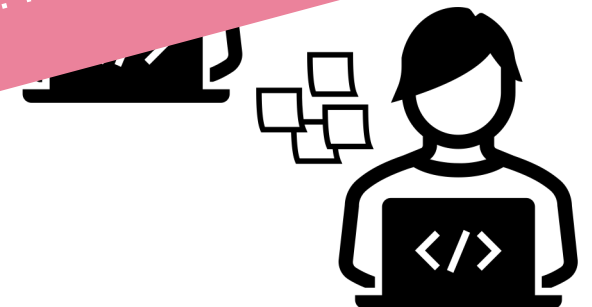
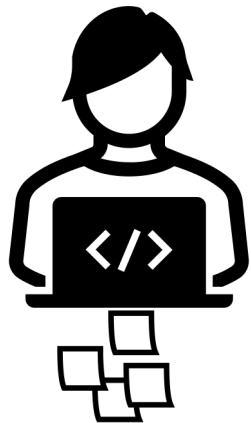
Lessons Learned:
Implement communication with TCP/UDP sockets in C. Build a simple application (e.g., producer-consumer pipeline) where containers communicate through exposed ports.

11. Key Logger Development

In this project, you will create a custom key logger program in C. Address two scenarios:

- Single machine with owner and guest to prevent inappropriate website access.
- Two machines to log keys on one machine and use the logged information on the other.

The project will cover system calls related to keyboard event monitoring, file I/O, and process hiding. This project introduces security aspects of system programming.



12. Trickery with LD_PRELOAD

LD_PRELOAD is an environment variable and a mechanism to preload a custom shared library into any program. This allows you to hijack C library (glibc) wrapper functions that programs use to interact with the OS. This can be leveraged for all sorts of trickery regarding custom functionality, process monitoring, and security.

```
[user]$ ./random_10
73 93 72 59 21 86 32 39 86 58

[user]$ ./random_10
55 63 41 81 39 86 72 63 29 66

[user]$ LD_PRELOAD=./hijack.so ./random_10
42 42 42 42 42 42 42 42 42 42

[user]$ export LD_PRELOAD=./not_random.so

[user]$ ./random_10
42 42 42 42 42 42 42 42 42 42
```



Lessons Learned:
Tip: use strace to identify which library functions a command uses before trying to hijack them.
Warning: functions often call other functions internally. Use wrapper variables to control when your hijacked versions are active, to avoid recursion.

13. Custom Text Editor

Develop a custom text editor in C, focusing on file operations and buffer management (open, close, save, copy, paste, find, replace, etc.). This project strengthens your understanding of system calls and provides practical experience in application development at the system level.



tiny



Text/Code
Editing



Web
Development

Lessons Learned:
You can use ncurses for terminal rendering. Consider your buffer data structure carefully (gap buffer, piece table, or rope) this choice significantly affects performance on large files.



File
Compare

14. Retro Game Console Emulator

Develop a simple emulator for a classic game console, focusing on system architecture and hardware emulation.

Written in C, it requires understanding of CPU emulation, memory management, and I/O handling, offering a fun way to explore system programming concepts.



15. Your Project Ideas?

You are free and encouraged to propose your own projects ideas and discuss them with us.

*Note: Your project must relate to **C** programming language and **OS concepts**.*



Overview

1	Supervisors
2	Timeline
3	Requirements
4	Templates and Guidelines
5	Project Ideas
6	Next Steps

Next Steps

Registration for projects is available on ADAM > Projects > Etherpad, first come first served basis!

March 6, 13:15 Introduction to project ideas

- Form teams of **four** students and choose a topic.
- Write a short project proposal (description and requirements).
- Hand-in your project proposal as PDF on ADAM.

March 13, 16:00 Submit project proposals on ADAM **[deadline]**

- Receive feedback and approval from project supervisors until **March 18**.
- Create your project idea presentation and upload it as PDF with the correct name on ADAM.

Have fun with your OS projects! Questions?